UNIVERSITATEA DE STAT DIN MOLDOVA

FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ

DEPARTAMENTUL INFORMATICĂ

Haratov Raul, IA2402

Lucrare individuală

la disciplina „Programarea în PYTHON”

Coordonator: _____ *Nume, prenume grad științific*

Chișinău, 2025

1. Scurtă prezentare a limbajului Python

Python este un limbaj de programare **interpretat, multi-paradigmă** (OOP, procedural, funcțional) și cu sintaxă concisă, publicat pentru prima dată în 1991 de Guido van Rossum. Popularitatea lui se datorează lizibilității codului, ecosistemului bogat de pachete și portabilității pe orice platformă majoră.

Domenii uzuale de utilizare:

- dezvoltare web;
- știința datelor și inteligență artificială (NumPy, Pandas, TensorFlow);
- automatizare și scripting;
- aplicații desktop și GUI (Tkinter, PyQt, Kivy);
- jocuri simple și prototipuri (Pygame);
- educație și cercetare experimentală.

2. Expunere succintă a problemei

Se dorește o **aplicație recreativ-educativă** care să simuleze jocul de cărți *Blackjack* (21). Proiectul servește drept demonstrație practică a abilității de a folosi Python pentru programare orientată pe obiecte, creare de interfețe grafice și gestionare multimedia (imagini + sunet).

3. Logica algoritmilor implementați

3.1 Generarea pachetului de cărți și amestecarea

```
suits  = ["hearts", "diamonds", "clubs", "spades"]
values =
["2","3","4","5","6","7","8","9","10","jack","queen","king","ace"]

deck = [Card(suit, value, f".../{value}_of_{suit}.png")
        for suit in suits for value in values]
random.shuffle(deck)  # amestecă pachetul
```

3.2 Distribuirea cărților / acțiunile *Hit* & *Stand*

- La *Start* sunt distribuite 2 cărți jucătorului și 2 dealer-ului (una acoperită).
- *Hit* → jucătorul primește încă o carte (`give_new_card()`), scorul se recalculează.
- *Stand* → controlul trece la dealer, care trage cărți până când scorul ≥ 17.

3.3 Calculul scorului şi gestionarea as-urilor

```
player_score = 0
aces = 0
for value in player_hand:
    if value.isdigit():
        player_score += int(value)
    elif value in {"jack","queen","king"}:
        player_score += 10
    else:  # atu
        aces += 1
for _ in range(aces):
    player_score += 11 if player_score + 11 <= 21 else 1
```

Această logică tratează **dinamic** valoarea as-ului (1 sau 11) pentru a evita depășirea pragului 21.

3.4 Determinarea rezultatului

- *Blackjack* sau **21 exact** → victorie instantă.
- Scor > 21 → înfrângere.
- La *Stand*, se compară scorurile; egalitatea afișează mesajul *Push*.

4. Interfața grafică (GUI)

- Bibliotecă: **Tkinter** (standard în CPython)
- Rezoluție fereastră: **800 × 600 px**
- Elemente vizuale:
    - Imagine de fundal cu masa de joc (BG_Table.png);
    - Cărți redimensionate la 80 × 116 px și plasate relativ pe masă;
    - Butoane *Hit* și *Stand* realizate ca `Label`-uri cu font *OCR-A*;
    - Ecrane dedicate pentru *You Win* / *You Lose* animate (`after()` + redimensionare).





5. Structuri de date utilizate

Obiect / variabilă	Tip	Rol în aplicație
`Card`	Clasă OOP	Incarnează o carte (semn, valoare, imagine)
`deck`	list[Card]	Pachetul de 52 de cărți, amestecat înainte de fiecare joc
`player_hand`, `dealer_hand`	list[str]	Valorile cărților deținute de fiecare participant
`all_cards`	list[tk.Label]	Referințe la widget-urile pentru a putea fi distruse la restart
`player_score`, `dealer_score`	int	Punctajele curente

6. Module și biblioteci suplimentare

Pachet	Funcționalitate	Motiv utilizare

`random`	funcția `shuffle`	Amestecarea pachetului
`tkinter`	GUI standard	Crearea și gestionarea ferestrei și a widget-urilor
`PIL.Image`, `PIL.ImageTk` (Pillow)	Prelucrare imagini	Redimensionare și conversie către format compatibil Tk
`pygame.mixer`	Redare audio	Muzică de fundal și efecte de *flip card* / *hit*

7. Descrierea funcționalităților

1. **Pornire** – utilizatorul apasă oriunde în fereastră ⇒ sunet de fundal + distribuirea inițială a cărților.
2. **Joc activ** – butoanele *Hit* / *Stand* devin interactive:
    a. *Hit* adaugă o carte jucătorului și verifică imediat condițiile de victorie/înfrângere.
    b. *Stand* dezvăluie cartea ascunsă a dealer-ului și rulează logica automată a dealer-ului.
3. **Ecran final** – mesaj animat *You Win* / *You Lose* (sau *Push*) + opțiunile *Try again?* și *Quit*.



8. Concluzii

Proiectul **demonstrează** modul în care Python poate integra:

- programare orientată pe obiecte (clasa `Card`),
- gestionarea interfețelor grafice (Tkinter),
- prelucrare multimedia (Pillow și Pygame),
- controlul fluxului și al evenimentelor (`bind`, `after`).

Datorită sintaxei accesibile, a bibliotecilor standard și externe bogate, Python a permis dezvoltarea rapidă a unui joc complet funcțional, cu interacțiune vizuală și auditivă.